If your existing application is complex and makes extensive use of the previous MVC or web API extensibility points or message handlers, porting your application to ASP.NET Core may be more difficult. ASP.NET Core is built with many features similar to the previous version of ASP.NET MVC, but the underlying architecture is different. Several of the previous features don't have direct replacements, so they'll require rethinking.

The larger the application is, the greater the difficulty you're likely to have converting your application to ASP.NET Core. Microsoft itself suggests that porting an application from ASP.NET MVC to ASP.NET Core is at least as big a rewrite as porting from ASP.NET Web Forms to ASP.NET MVC. If that suggestion doesn't scare you, nothing will!

If an application is rarely used, isn't part of your core business, or won't need significant development in the near term, I suggest that you *don't* try to convert it to ASP.NET Core. Microsoft will support .NET Framework for the foreseeable future (Windows itself depends on it!), and the payoff in converting these fringe applications is unlikely to be worth the effort.

So when *should* you port an application to ASP.NET Core? As I've already mentioned, the best opportunity to get started is on small new greenfield projects instead of existing applications. That said, if the existing application in question is small or will need significant future development, porting may be a good option.

It's always best to work in small iterations if possible when porting an application, rather than attempt to convert the entire application at the same time. Luckily, Microsoft provides tools for that purpose. A set of System.Web adapters, a .NET-based reverse proxy called YARP (Yet Another Reverse Proxy; http://mng.bz/qr92), and tooling built into Visual Studio can help you implement the strangler fig pattern (http://mng.bz/rW6J). This tooling allows you to migrate your application